

Software Engineering

INSTRUCTOR : DR. NATASH ALI MIAN

School of Computer and Information Technology

Beaconhouse National University

Software Design

Converting to diagrams

Converting the requirements in a form that is ready for the
implementation

Architecture of a house

A solid orange horizontal bar at the bottom of the slide.

Design?

Synonyms: plan, arrangement, lay out, map, scheme

Antonyms: accident, fluke, chance, guess

Design is an activity of creating a solution that satisfies a specific goal or need

Design is the backbone of all products and services

Considered an artistic and heuristic activity

Software Design

Deriving a solution which
satisfies software
requirements

Stages of Design

Problem understanding

- Look at the problem from different angles to discover the design requirements.

Identify one or more solutions

- Evaluate possible solutions and choose the most appropriate depending on the designer's experience and available resources.

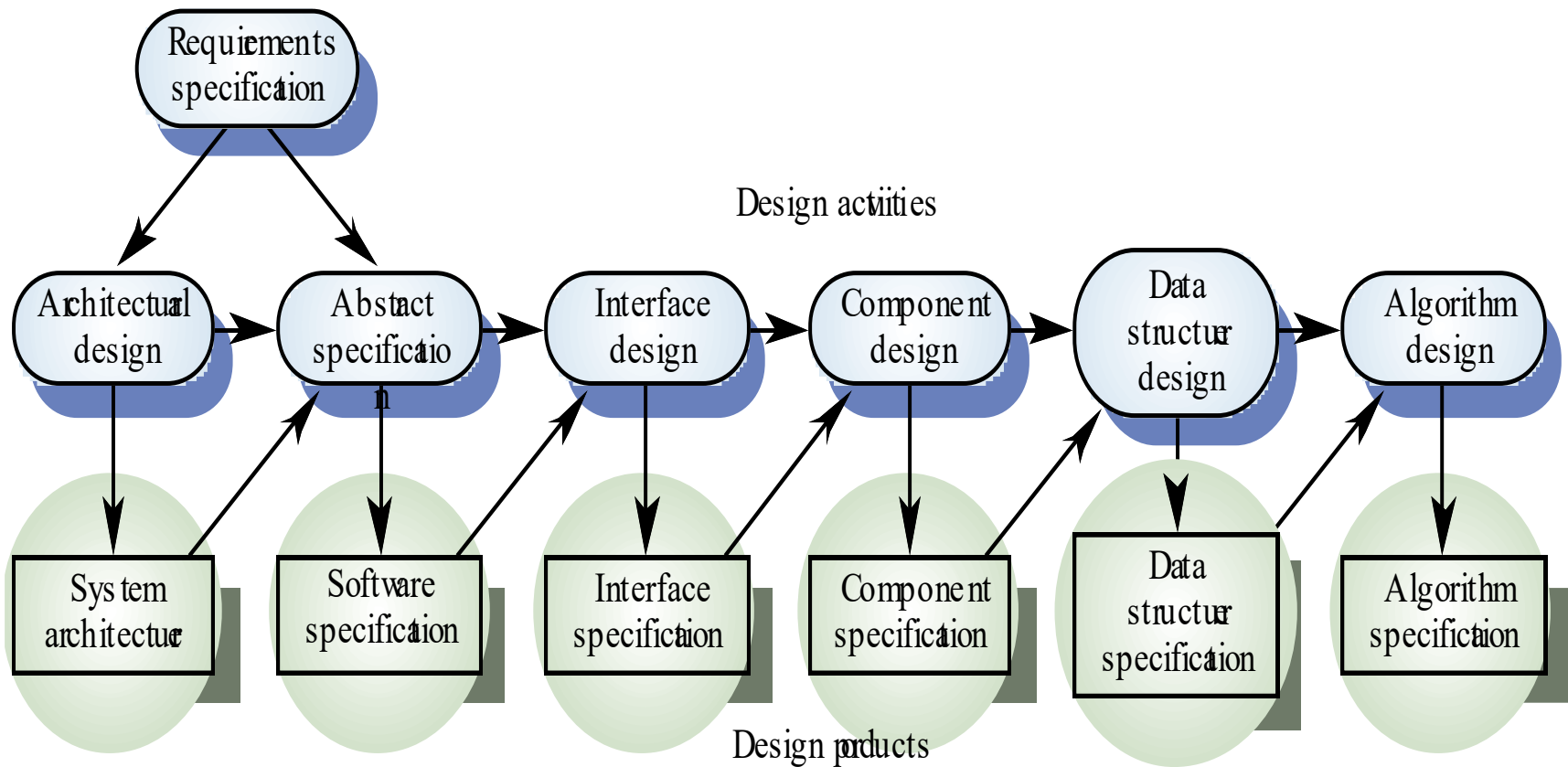
Stages of Design

Describe solution abstractions

- Use graphical, formal or other descriptive notations to describe the components of the design.

Repeat process for each identified abstraction until the design is expressed in primitive terms.

Phases in the Design Process



Design Phases

- ***Architectural design:*** Identify sub-systems.
- *Abstract specification:* Specify sub-systems.
- *Interface design:* Describe sub-system interfaces.
- *Component design:* Decompose sub-systems into components.
- *Data structure design:* Design data structures to hold problem data.
- *Algorithm design:* Design algorithms for problem functions.

Design

Computer systems are not monolithic: they are usually composed of multiple, interacting modules.

Modularity has long been seen as a key to cheap, high quality software.

The goal of system design is to decode:

- What the modules are;
- What the modules should be;
- How the modules interact with one-another

Cost X :It is already defined and finalized

Sol 1

1 S

4 modules

16 sub modules

64 components

256 sub components

Sol 2

1 S

4 modules

16 sub modules

Sol 3

1 S

4 modules

16 sub modules

64 components

Sol 4

1 S

4 modules

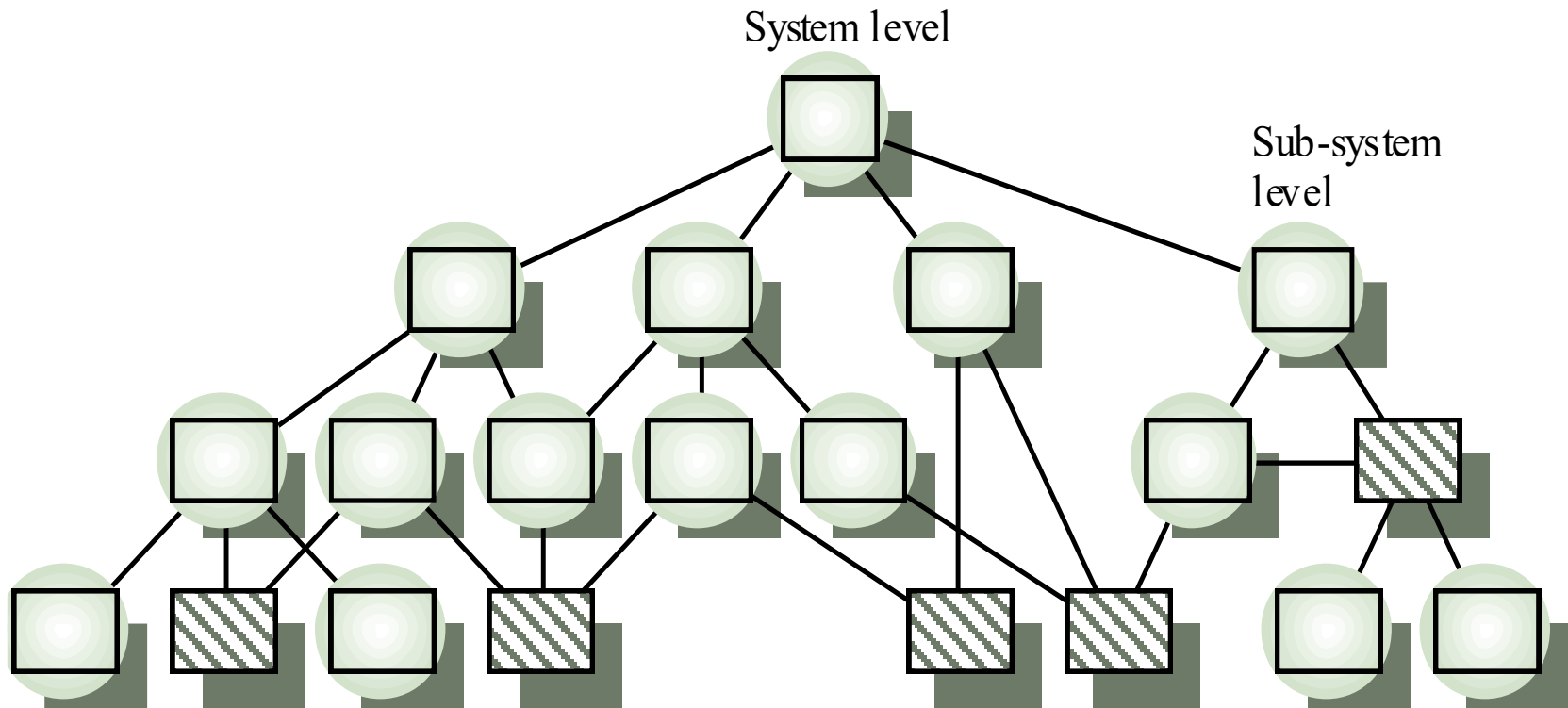
16 sub modules

64 components

256 sub components

512 sub components

Hierarchical Design Structure



Software Design - 1

Software design is an artifact that represents a solution, showing its main features and behavior, and is the basis for implementing a program or collection of programs

Software Design - 2

Design is a meaningful representation of something that is to be built. It can be traced to a customer's requirements and at the same time assessed for quality against a set of pre-defined criteria of "good" design

Software Design - 3

Software design is different from other forms of design because it is not constrained by physical objects, structures, or laws

As a result, software design tends to be much more complex than other forms of design because it is conceptually unbounded, whereas the capabilities of the human mind are bounded

Without Software Design

We risk building an unstable system

- one that will fail when small changes are made
- one that may be difficult to test
- one whose quality cannot be assessed until late in the software process
- one that will be of no or very little use for similar projects (not reusable)

Design Process and Model

SOFTWARE DESIGN IS BOTH A PROCESS AND A
MODEL

Design Process

It is a sequence of steps that enables a designer to describe all aspects of the software to be built

During the design process, the quality of the evolving design is assessed with a series of formal technical reviews or design walkthroughs

Needs creative skills, past experience, sense of what makes “good” software, and an overall commitment to quality

Design Model

Equivalent to an architect's plan for a house

Represents the totality of the thing to be built

Provides a variety of different views of the computer software

Design Defects

Design Defects - 1

Defects introduced during **preliminary design** phase are usually not discovered until **integration testing**, which is too late in most cases

Defects introduced during **detailed design** phase are usually discovered during **unit testing**

Design Defects - 2

All four categories of defects are found in design models

- Errors of commission (Something Wrong)
- Errors of omission (Something left out)
- Errors of clarity and ambiguity (Multiple interpretations of same thing)
- Errors of speed and capacity (capability or shortage of time)

Code:

Application is working but not fast enough

Variable Limit (Error, limit cross)

Design Process

Design Process

The design process produces

- an architectural design
- a component design
- an interface design
- a data design

Data Design

The data design transforms the information domain model created during analysis into the data structures that will be required to implement the software

Architectural Design

It defines the relationship between major structural elements of the software. Architectural design representation is derived from system specification, analysis model, and interaction of subsystems

Interface Design

Interface design describes how the software communicates within itself, with systems that interoperate with it, and with humans who use it. An interface implies a flow of information

Component Design

Component-level design transforms structural elements of software architecture into a procedural description of software components

Think the Right Way

To achieve a good design, people have to think the right way about how to conduct the design activity

- Katharine Whitehead

Design Process Principles - 1

The design process should not suffer from “tunnel vision”

The design should not reinvent the wheel

The design should “minimize the intellectual” distance between the software and the problem as it exists in the real world

Design Process Principles - 2

The design should exhibit uniformity and integration

The design should be assessed for quality as it is being created, not after the fact

The design should be reviewed to minimize conceptual (semantic) errors

When Applying them

Plan for change, because it is expected

Plan for failure, because no software system is free of defects

How do we know if the design we have developed is of high quality?

Design Process Evaluation Guide # 1

The design must implement all of the explicit requirements contained in the analysis model, and it must accommodate all of the implicit requirements desired by the customer

Design Process Evaluation Guide # 2

The design must be readable and understandable guide for those who generate code, write test cases, and test the software

Design Process Evaluation Guide # 3

The design should provide a complete picture of the software, addressing the data, functional, and behavioral domains from an implementation perspective

Design Model/Representation

Design Model Principles - 1

Separation of concerns

Modeling real-world objects

Minimizing the interactions among cohesive design components

The design should be traceable to the analysis model

Design Model Principles - 2

The design should be structured to accommodate change

The design should be structured to degrade gently, even when aberrant data, events, or operating conditions are encountered

Design is not coding, coding is not design

Guidelines for Good Design Model - 1

A design should exhibit an architectural structure that

- Has been created using recognizable design patterns
- Is composed of components that exhibit good design characteristics
- Can be implemented in an evolutionary fashion, facilitating implementation and testing

Guidelines for Good Design Model - 2

A design should be modular; that is software should be logically partitioned into elements that perform specific functions and sub-functions

The design should contain distinct representations of data, architecture, interfaces, and components (modules)

Guidelines for Good Design Model - 3

A design should lead to data structures that are appropriate for the objects to be implemented and are drawn from recognizable data patterns

A design should lead to components that exhibit independent functional characteristics

Guidelines for Good Design Model - 4

A design should lead to interfaces that reduce the complexity of connections between modules and with external environment

A design should be derived using a repeatable method that is driven by information obtained during software requirements analysis

Abstraction

Abstraction permits one to concentrate on a problem at some level of generalization without regard to irrelevant low-level details

Abstraction is one of the fundamental ways that we as humans cope with complexity

Grady Booch

Levels of Abstraction - 1

At the highest level of abstraction, a solution is stated in broad terms using the language of the problem environment

At lower levels of abstraction, a more procedural orientation is taken. Problem-oriented terminology is coupled with implementation-oriented terminology in an effort to state a solution

Levels of Abstraction - 2

At the lowest level of abstraction, the solution is stated in a manner that can be directly implemented

Types of Abstraction - 1

Procedural abstraction

- Named sequence of instructions that has a specific and limited function
- Example: Open door

Data abstraction

- Named collection of data that describes a data object
- Example: any object

Types of Abstraction - 2

Control abstraction

- Implies a program control mechanism without specifying internal details
- Example: synchronization semaphore

Refinement - 1

A program is developed by successively refining levels of procedural detail

A hierarchy is developed decomposing a macroscopic statement of function in a step-wise fashion until programming language statements are reached

Refinement is actually a process of elaboration

Refinement - 2

There is a tendency to move immediately to full detail, skipping the refinement steps. This leads to errors and omissions and makes the design much more difficult to review. Perform stepwise refinement

Abstraction and refinement are complementary concepts

Modularity - 1

One of the oldest concepts in software design

Software is divided into separately named and addressable components, often called, modules, that are integrated to satisfy problem requirements

Modularity - 2

Modularity is the single attribute of software that allows a program to be intellectually manageable

Don't over modularize. The simplicity of each module will be overshadowed by the complexity of integration

Quality Design Concepts

Information Hiding - 1

Modules should be specified and designed so that information (procedures and data) contained within a module is inaccessible to other modules that have no need for such information

IH means that effective modularity can be achieved by defining a set of independent modules that communicate with one another only that information necessary to achieve a software function

Information Hiding - 2

Abstraction helps to define the procedural (or informational) entities that make up the software

IH defines and enforces access constraints to both procedural detail within a module and any local data structure used by the module

Cohesion - 1

Cohesion is the qualitative indication of the degree to which a module focuses on just one thing

In other words, cohesion is a measure of the relative functional strength of a module

A cohesive module performs one single task or is focused on one thing

Cohesion - 2

Highly cohesive modules are better, however, mid-range cohesion is acceptable

Low-end cohesion is very bad

Coupling

Coupling is a qualitative indication of the degree to which a module is connected to other modules and to the outside world

In other words, coupling is a measure of interconnection among modules in a software structure

Loose coupling is better. Simple connectivity is easier to understand and less prone to “ripple effect”

Design Methods

Use a design method, which is most suitable for the problem at hand.
Don't just use the latest or the most popular design method

There are many structured design and object-oriented design methods to choose from.

Severity Levels

Types of Errors

4 levels

Level 0 – Cosmetic Error	1	4
Level 1 – Minor Errors	2	3
Level 2 – Major Errors	3	2
Level 3 - Total Application Crash	4	1

Sources of Errors

Requirement Errors

Design

Coding

Test Cases

Deployment

Maintenance

Types of Error

Sources of Error

Severity

